

Speaking script — WireGuard for Apache NuttX

(Community Over Code, Glasgow)

Deck: [coc-glasgow-slides.html](#) (and its official-template edition [coc-glasgow-reveal/](#)), 41 slides, English.

Generated by [export_en_script.py](#) from the speaker notes in the deck (press **N** there to see them live): the "Say" text below *is* those notes. Do not edit this file by hand. Edit the notes in the deck and re-run the script, or the two will drift. The Japanese [rehearsal script](#) follows the same slide order.

Total: about 33:22, including a 90-second demo. Times are cumulative ("you should be at roughly this point") at 150 words per minute. That is already a little over a 30-minute slot, so rehearse to land the main body at 27–28 minutes and keep the cut list below ready.

The centre of gravity is slides 31–33 (verification): together they are 9.8 minutes of the talk, and they carry the claims a reviewer will actually check. Do not compress that block to protect time elsewhere. Cut before it or after it, never inside it. They are one argument: the first is where it runs, the second is what "it runs" does not prove, the third is the one place the answer is a judgment call rather than a test. Splitting them loses the argument.

Do not cut slides 4–14 either (About Me through Architecture). What NuttX is, where it is used, Sony's history with it, where I met it and why that led to WireGuard: everything after depends on that chain.

If time is short, drop in this order:

1. Slide 39 (Contributing Back). It supports the talk but is not its spine; if dropped, answer it in Q&A if asked.
2. Slide 38 (portability table). Supporting evidence, not the spine.
3. Slide 37 (operability).

If the live demo stalls, switch to the recording at once. **The agenda dividers (3, 15, 19, 26, 30, 34, 36) are a ten-second breath each:** point at the highlighted line and pause, do not read the list. With no time at all, advance through them silently. They save almost nothing, so cut the three slides above instead.

Check before the talk: the GitHub star counts on slide 8 change. Re-check them on the day, or leave the date on the slide.

1 — Title (0:00 → 0:45)

On slide: "WireGuard for Apache NuttX." ESP32-S3 & SPRESENSE tunnel over real Wi-Fi · real WireGuard peers (Linux kernel, Windows client) · runs in a real kernel build (BUILD_KERNEL) · one source across NuttX 13.0.1 / master / 12.7.0.

Say:

- Today I will talk about bringing WireGuard to Apache NuttX.

- It is a WireGuard VPN implemented as a wg0 network device. The tunnel is up over real Wi-Fi on ESP32-S3 and on SPRESENSE, and the peers are always real WireGuard implementations — the Linux kernel module and the official Windows client. The apps version builds from one source on three NuttX versions. Later in the talk the new driver also runs inside real KERNEL and PROTECTED builds, which is the harder and more interesting half.
- But the point of this talk is not that it works. It is what "works" hid, and what pushing it into the kernel forced into the open.

2 — Agenda (0:45 → 1:18)

On slide: The seven parts as sentences with their times: why I needed a VPN on a NuttX device (~10 min) / how I reused an existing WireGuard (~2) / bugs that a working demo hid (~4) / moving the driver into the kernel (~2) / how I checked that it really works (~10) / using a real board through the tunnel (~2) / what comes next (~3).

Say:

- Here is the shape of the talk.
- Seven chapters. Background — what NuttX is, where it's used, Sony's history with it, my own path into it, and the problem that led to WireGuard — then the porting strategy that started from an existing lwIP implementation, five hidden pitfalls a working demo never revealed, moving the driver into the kernel, how it was verified, a short demo, and then what to carry out and where this goes upstream. About thirty minutes, then questions.

3 — Agenda: ① Background (1:18 → 1:28)

Say (one line): Here we go — chapter one.

Pause: point at the highlighted line, one beat.

4 — About Me (1:28 → 2:11)

On slide: Photo, and a timeline Robotics → Computer Vision → Edge AI → Embedded Systems → NuttX / WireGuard with a picture per step.

Say:

- Quickly, who is talking.
- I am an edge AI engineer at Sony Semiconductor Solutions, working with Apache NuttX from the application side. I was born in Nagoya, went to university in Sendai where I studied space, robotics, MEMS and semiconductors, and am based in Kawasaki now. I go to a lot of hackathons, help organize software conferences, and have a book on open source out as well.
- The path that got me into this specific project runs through molecular robotics, computer vision and AI systems work, edge AI cameras, and embedded systems on SPRESENSE — NuttX and WireGuard are the latest stop on that path, not the start.

5 — Project Overview (2:11 → 2:54)

On slide: The wg0 netdev picture: application / NSH, the NuttX network stack, wg0 next to eth0 / wlan0, a UDP socket underneath, the Internet, the peer.

Say:

- Here is the whole idea in one picture.
- Apache NuttX is a POSIX-compliant RTOS with its own TCP/IP stack. It had no VPN. This project adds wg0 as a virtual network interface: applications see an ordinary interface, and underneath, the encrypted transport is a UDP socket. The peer on the other end is a real WireGuard endpoint — Linux, Windows, or another gateway.
- That POSIX and BSD-socket shape is why NuttX is a good fit for this work. This is not a claim that FreeRTOS or Zephyr cannot run a VPN; it is a claim about how naturally a VPN can become an ordinary network device here.

6 — How It Started: February to October (2:54 → 3:44)

On slide: A month axis Feb → Oct. Above it: mid Feb, read the GSoC idea list / early Mar, plan written up as an issue / Mar 20, heard about this conference from Alan, sent a talk / early Apr, GSoC proposal / early May, talk accepted, GSoC not selected / Oct, Glasgow. Below it: Jun–Jul a little at a time, August holiday most of the work, September FLAT app → kernel driver.

Say:

- Here is how this project happened, month by month.
- In the middle of February I read the GSoC idea list and started to study. In early March I wrote my plan as an issue and proposed it. On March 20, Alan told me about this conference, so I sent a talk. In early April I sent my GSoC proposal.
- In early May I got two answers: the talk was accepted, and the GSoC proposal was not. I decided to build it anyway.
- From June I worked on it a little at a time. Most of the work happened in the summer holiday in August. In September I moved it from a FLAT app into a kernel driver — that is the second half of this talk.

7 — Background: What Is NuttX (3:44 → 4:33)

On slide: Software stack (applications / POSIX, ANSI, BSD socket API / networking, VFS, file systems, device drivers, NSH / NuttX kernel / MCU, MPU) and tags: RTOS, POSIX-oriented, BSD sockets, VFS, NSH, native networking, device drivers, ASF top-level project.

Say:

- Before anything else: what NuttX actually is.
- NuttX is a real-time operating system built around a standards-first promise. It is POSIX and ANSI-API oriented, it speaks BSD sockets for networking, it has a real filesystem and VFS layer, a shell

called NSH, native networking, and a device-driver model — all scaled down to run on microcontrollers from 8 to 64 bit. It is also an Apache Software Foundation top-level project, developed in public by its contributor community.

- The point I want to land here is narrow but important: NuttX is a small RTOS with a Unix-like programming model. That combination is unusual, and it is why almost everything later in this talk — sockets, netdevs, a POSIX-shaped port — is possible at all.

8 — Positioning: Where NuttX Fits (4:33 → 5:29)

On slide: FreeRTOS / Zephyr / NuttX table (design centre, typical use), GitHub stars with the note that stars are visibility, not deployment; number cards: 15+ architectures, 300+ boards, 1500+ configs, Sony, PX4, Xiaomi OpenVela 1000+ SKUs, Japan's 2024 lunar mission.

Say:

- Thirty seconds on where NuttX sits, and whether it is a toy.
- Three capable RTOSes, three different centers of gravity. FreeRTOS is a deliberately small real-time kernel for MCU firmware and control. Zephyr is an integrated platform for connected devices, wearables and industrial IoT. NuttX is the Unix-like one: networking, filesystems, drivers, used in drones and sensing. Not a ranking — a shape. By GitHub stars NuttX is the smallest of the three, and I want to say that plainly rather than hide it; stars are developer visibility, not deployment volume.
- And the deployment side is real: fifteen-plus architectures, three hundred-plus boards, fifteen hundred-plus configuration templates upstream. In production it runs Sony devices, PX4 flight controllers, more than a thousand Xiaomi OpenVela SKUs, and it flew on Japan's 2024 lunar mission. Niche next to the giants, not a toy project.

Check before the talk: the star counts move. Re-check them on the day, or leave the date on the slide as it is.

9 — History: NuttX at Sony (5:29 → 6:36)

On slide: Timeline: 2015 Sony audio products / 2018–19 SPRESENSE, CXD56xx (OSS + ELC Europe 2019) / 2020 upstream collaboration (NuttX Online Workshop, "A Journey from Fork to Mainline") / 2020s sensing and edge devices.

Say:

- This is Sony's history with NuttX, not mine yet — and it is longer than edge AI.
- Sony has shipped audio products running NuttX since around 2015. SPRESENSE and its CXD56xx chip, introduced a few years later, brought audio, GNSS, camera and multicore support into NuttX, and Masayuki Ishikawa from Sony presented that work at Open Source Summit plus Embedded Linux Conference Europe in 2019. From there, Sony engineers — including Alin Jerpelea, who later chaired the NuttX project management committee — became significant upstream contributors; his 2020 talk was literally titled "A Journey from Fork to Mainline."
- I am not claiming one engineer caused Sony's adoption of NuttX — I don't have a source for that, so I won't say it. What I can say from public talks and the docs is simpler: Sony's NuttX usage predates

edge AI by the better part of a decade, and it broadened over time into camera, GNSS and sensing work. That is the ground my own experience stands on.

10 — Experience: SPRESENSE and AITRIOS (6:36 → 7:30)

On slide: Photos of SPRESENSE and the AITRIOS edge AI camera; the flow "application development → system integration → deployment & operation".

Say:

- That was Sony's history. This is mine — same hardware family, a different vantage point.
- On SPRESENSE, I work with the NuttX SDK for sensing, camera, GNSS and LTE work — prototyping and embedded development, application side. On AITRIOS edge AI cameras, built around ESP32 and NuttX, the work is about getting a camera deployed and operating in the field, not sitting on a bench next to a debugger.
- The common thread is the direction I approach NuttX from: application development, then system integration, then deployment and operation. I never had to write a NuttX driver to do any of that — until this project. And the problem that got me there was not "how do I get NuttX to run." It was "how do I reach a NuttX device after it has already been deployed."

11 — Problem: Reaching a Device From Far Away (7:30 → 8:18)

On slide: Engineer → Internet → router → Wi-Fi → NuttX device, showing that WPA2/WPA3 covers only the Wi-Fi hop; cards: open it to the Internet / make your own protocol / use a vendor's cloud.

Say:

- Now the problem itself, stated plainly.
- Wi-Fi security is real, but it protects one hop: the radio link between the device and the access point. Everything before that — the engineer, the internet, the router — is outside what WPA2 or WPA3 can promise. What I needed was authentication and encryption for the whole path, engineer to device, not just the last few meters of it.
- Without that, the practical options are all worse: expose the device with a public IP and hope, invent and maintain a bespoke secure protocol, or hand the connection to a vendor's cloud. The same gap shows up in edge AI, industrial IoT, and remote infrastructure generally — it is not specific to this one project.

12 — Options: Other Ways to Reach a Device (8:18 → 9:33)

On slide: A table of seven ways (open a port / HTTPS to a cloud API / MQTT broker / reverse tunnel or relay / VPN on the router / carrier closed network / VPN on the device) against: who connects, any service or not, server in the middle, where security ends, what NuttX already has.

Say:

- Before I chose WireGuard, I compared the usual ways to reach a device.

- Opening a port is easy, but then the whole Internet can reach it. HTTPS and MQTT are what most IoT products use. The device connects out to a server, so NAT is not a problem, and NuttX already has clients for both. But you only get the messages you designed — you cannot log in or open a web page on the device. And the security ends at the broker or the cloud.
- A reverse tunnel gives you one port through a relay server. A VPN on the router is great when the router is yours — but the last hop on the LAN is open, and at a customer site the router is usually not yours. A carrier closed network works only over cellular and needs a contract.
- A VPN on the device is the only row that gets all of it: any IP service, no server in the middle, and security all the way to the device. MQTT and HTTPS are still the right tools for data. This is for maintenance access.

If asked about Tailscale or ZeroTier: they are also a VPN on the device, but they usually need a bigger OS than a microcontroller has today (footnote on the slide).

13 — Solution: Why WireGuard (9:33 → 10:23)

On slide: Engineer / Linux / Windows ↔ WireGuard tunnel ↔ NuttX device; modern IP-layer VPN / UDP transport, public-key peers / already everywhere.

Say:

- Given that problem, why WireGuard specifically.
- WireGuard is a modern, IP-layer VPN: small and simple by design, a tiny fraction of the code size of IPsec or OpenVPN, which matters a lot on a microcontroller. It runs over plain UDP, and peers are identified by public keys rather than certificates or a negotiated session. And critically, it is not exotic: it has been in the mainline Linux kernel for years, there is an official Windows client, and the other end of the tunnel can be any existing WireGuard peer.
- So the choice was not "adopt a product." It was "adopt a standard protocol instead of inventing a bespoke one or leaning on a vendor's cloud." That is what makes it fit this specific problem.

14 — Architecture: WireGuard on the NuttX Network Stack (10:23 → 11:08)

On slide: Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0 / usrsock → network, with TX and RX arrows.

Say:

- One level more technical than the overview: this is where implementation starts.
- Outbound, a packet comes down from the application through TCP or UDP over IP, reaches wg0 like any other netdev, gets encrypted by WireGuard, and leaves as a UDP datagram over wlan0 on a board, or over usrsock where Wi-Fi is itself a socket-based peripheral. Inbound traffic is the same path in reverse: a UDP datagram arrives, WireGuard decrypts it, and it is handed back up to wg0 as if it had arrived on an ordinary interface.
- Everything from here is about building that middle section — wg0 and the encryption step — without touching the layers above or below it.

15 — Agenda: ② Porting Strategy (11:08 → 11:18)

Say (one line): Quick check-in: background is done, porting strategy is next.

Pause: point at the highlighted line, one beat.

16 — Porting Strategy: Reusing wireguard-lwip Without lwIP (11:18 → 12:04)

On slide: Three cards: protocol & crypto (KEEP) / platform hooks (ADAPT, the four functions in wireguard-platform.h) / lwIP network glue (REPLACE).

Say:

- Why start from smartalock/wireguard-lwip instead of writing a crypto stack from scratch.
- wireguard-lwip splits cleanly into three layers. The protocol and crypto — handshake, transport, BLAKE2s, ChaCha20-Poly1305, Curve25519 — are portable C with no OS dependency, so they come across untouched. All OS-specific behaviour was already isolated behind one header, wireguard-platform.h, as four functions. Only the lwIP network glue, the part that registers an lwIP netif and manages lwIP buffers, needed to be replaced, because NuttX does not have lwIP at all.
- The fact that I didn't have to rewrite anything is not luck — it's what a clean platform boundary buys you. That boundary is the reason this port was weeks, not months.

17 — Design: Mapping lwIP onto NuttX netdev (12:04 → 12:24)

On slide: netif → net_driver_s, pbuf → iob, netif_add() → netdev_register(), callbacks → devif_poll().

Say:

- The single most important porting decision.
- Rather than drag lwIP into NuttX, I mapped the abstractions onto NuttX's own. netif becomes net_driver_s, pbuf becomes iob, netif_add becomes netdev_register, and the send callback becomes NuttX's devif_poll. wg0 is registered as a TUN-type netdev, and the "wire" underneath it is a UDP socket.

18 — Result: A Working Tunnel Is Only the Beginning (12:24 → 12:51)

On slide: Protocol & crypto: 3,079 lines unchanged / OS hooks: four functions / network glue: the real work. wg genkey / set / setconf at run time.

Say:

- The result of the first version.
- The protocol and crypto came across byte-identical — over three thousand lines unchanged. The four OS hooks are small. The real work was the network glue. And it worked: you can generate a key,

set peers, load a config, all at run time, no rebuild. So far, a success story. The rest of the talk is about what that success hid.

19 — Agenda: ③ Hidden Pitfalls (12:51 → 13:01)

Say (one line): Background and porting strategy are done. Now the part of the talk this whole thing is organized around.

Pause: point at the highlighted line, one beat.

20 — The Testing Pattern: Simple Tests Pass, Deep Tests Fail (13:01 → 13:28)

On slide: SIMPLE TEST (handshake / ping, PASS) → ASSUMPTION ("that part must be fine") → DEEPER TEST (TCP / real hardware / kernel / security, FAIL).

Say:

- This is the heart of the talk.
- Every difficult bug in this project had the same shape. A shallow test passes; a deeper one fails. Worse, the passing shallow test convinces you that area is fine, so you search everywhere else for hours. The next slides are instances of that pattern — five in the apps version, and then two more when I pushed it into the kernel.

21 — Pitfall 1/7: SO_RCVTIMEO (13:28 → 13:51)

Say:

- The first taste: a success return with no effect.
- Setting a receive timeout returned zero, success. But `recvfrom` still blocked forever — the option was accepted and silently ignored. That produced a perfect deadlock: the timer never fired, so no initiation was sent, so the peer never replied, so `recvfrom` never returned to let the timer fire.

22 — Pitfall 2/7: The Detached pthread (13:51 → 14:30)

Say:

- The second: a lifecycle assumption that did not hold.
- The receive thread was created detached. When the launching command exited, the thread stopped — it did not even show up in `ps`. A short sleep after creation let its `printf` run, so it really was created. My mistake was reading `detach` as "it will outlive the command" — `detach` only controls when the thread's resources are reclaimed, not whether it survives its launcher exiting. So when the command exited, the thread went with it. That is why the kernel version owns the thread in the driver instead.

23 — Pitfall 3/7: ping Works, TCP Dies (14:30 → 15:19) (*the good one*)

Say:

- My favourite bug in the project.
- On the same tunnel, ping was perfect and even a TCP handshake completed — then the connection died. Instrumenting the send path showed EBADF, bad file descriptor. The sends that worked all happened inside the receive task itself — handshake replies, keepalives, the ping echo, the SYN-ACK. Sends from any other task failed.
- The cause: in NuttX, file descriptors are scoped to the task group, not to the whole system, and not global. The fix was to stop going through fds entirely and use NuttX's internal socket API, which operates on a struct socket — just memory — usable from any task. This is exactly why the kernel version holds the socket as a struct socket.

24 — Pitfall 4/7: "No Real Impact" Was 10× (15:19 → 15:53)

Say:

- The fourth: an assumption I did not measure.
- The fd-less fix also broke poll, which is fd-based, so as a stopgap I polled the socket every fifty milliseconds. I assumed the only cost was waking the CPU when idle. Wrong. Every arriving packet waited up to fifty milliseconds, and that alone capped throughput — about ten times slower than it should be. I measured "no impact" and it was ten x. The real fix was a proper blocking wait: a callback on psock_poll and a semaphore.

25 — Pitfall 5/7: sim Passes, Hardware Freezes (15:53 → 16:27)

Say:

- The fifth, and it comes back later: stacks nobody measured.
- Persisting the config by redirecting into a file froze the board, but only on real hardware's flash filesystem, never in sim. The cause was the receive task's stack. It calls into the IP stack on its own stack, so the whole TCP call depth lands there, and for packets it answers immediately, the reply encryption too. The default 3072 bytes was a number nobody had measured. Stack coloration plus ps found it. Hold that thought.

26 — Agenda: ④ Kernel Driver (16:27 → 16:37)

Say (one line): Five pitfalls down. Now the harder half: making this a real kernel driver.

Pause: point at the highlighted line, one beat.

27 — Part 3: Into the Kernel (16:37 → 17:09)

Say:

- This is the additional work since the first version, and the part that makes it mergeable.
- Version one lives in apps and reaches into socket internals. That is fine for a flat build, but it can never be a proper in-tree driver, and it does not fit the protected and kernel builds where user space and kernel are separated. So I moved the device into the kernel, under drivers/net/wireguard, and drove it from a tiny user-space wg command through ioctl.

28 — The Kernel Design (17:09 → 17:50)

Say:

- The shape of the real driver.
- wg0 is now a lower-half netdev in the kernel, owning the UDP socket and a receive kernel thread — so the socket-ownership and launcher-lifetime problems from part two go away by design. Configuration is a flat, pointer-free ioctl ABI, pointer-free because the kernel copies the caller's structure directly. The private key is write-only: the get ioctl never hands it back — though it still lives in the config file on the user side. The kernel-side crypto is NuttX's own; the only vendored code is a small MIT X25519 in the user-space command, for offline key generation.

29 — Two More Bugs, Same Shape (17:50 → 18:49) (*locked with 26*)

Say:

- And the same pattern held at kernel depth — twice.
- First, a bug in NuttX's own ChaCha20-Poly1305: the counter went into the wrong bytes of the nonce. For counter zero the two layouts are identical, so the handshake completes and then every data packet after the first fails to decrypt. Shallow passes, deep fails. There was no in-tree caller, so it had never been exercised. A two-line fix, and it is going upstream as its own crypto PR — porting into the project found a latent bug in the project.
- Second, a crash that only happened in a kernel build. Setting the key snapshotted the peers into a six-kilobyte array on the stack, and the kernel stack is three kilobytes — overflow, heap corruption, panic. On sim, with large task stacks, it never showed. Exactly the stack lesson from part two, one level deeper. Moved to the heap.

30 — Agenda: ⑤ Verification (18:49 → 18:59)

Say (one line): The driver exists. Now, how do I know it's actually correct.

Pause: point at the highlighted line, one beat.

31 — Verification: Seven Places It Has to Hold (18:59 → 22:47) (*core — do not cut*)

On slide: Table: sim / rv-virt knetnsh64 (BUILD_KERNEL) / rv-virt pnsh64 (BUILD_PROTECTED) / rv-virt knetnsh64_smp (4 CPUs) / SPRESENSE measured / ESP32-S3 + SPRESENSE over real Wi-Fi / apps v0.1.1 on

both boards.

Say:

- Be precise about which implementation was verified where.
- The kernel driver builds clean in the sim config — the configuration CI compiles — and it runs as a full tunnel in a real kernel build: rv-virt's knetnsh64 in QEMU, over virtio-net to Linux kernel WireGuard, with the wg command loaded as a separate ELF and the driver reached across the syscall boundary.
- And then the other privilege model. The ioctl ABI is a fixed struct with no pointers inside it, so that the kernel never has to reach into user memory — and the way an assumption like that fails is not a compile error, it is an MPU exception on the first call. PROTECTED had only ever been built, on a board with no emulator. So I brought it up on rv-virt's protected config: the split boots, the wg command lives in the user blob, every configuration ioctl crosses the boundary, and the tunnel reaches Linux kernel WireGuard with no exception, no assertion, no panic. The memory map is the stock one — the kernel takes eighty percent of its side, the user command eight percent of the other.
- And the locking. The driver puts its protocol state under the device lock, not under there being only one CPU — and until this week that was an argument I made from the code rather than something I had contended. So: four CPUs, a two-thousand-packet flood arriving while another CPU is running configuration ioctls and a down-up cycle. No exception, no assertion, and the tunnel came back. One scripted interleaving on an emulator is not a race search, so a race that needs a specific window can still be hiding. But the claim is no longer only on paper.
- Last row, and the one I would put in front of a reviewer first. The sim soak said the resources come back exactly to where they started. Fine, but the simulator's allocator is the host's and its frames are sixty-four bit, so none of the heap numbers transfer to a board. So I measured the board: a hundred and twenty seconds of load, then forty down-up cycles, each one closing and reopening a socket that belongs to the Wi-Fi module through usrsock and re-handshaking over real Wi-Fi. Heap, allocation count, buffer pool, every stack high-water: identical before and after. And the number I actually wanted, the receive thread's deepest observed use, is one thousand four hundred and seventy-two bytes against a six-kilobyte default. That default used to be a guess. It is a measurement now.
- And it now runs on real hardware — both boards. The kernel driver flashed to an ESP32-S3, tunnelling to the official Windows WireGuard client over real Wi-Fi; and to a SPRESENSE, whose GS2200M Wi-Fi is a usrsock device, so the driver's sends really do go out through usrsock on hardware. And then the fault case that design exists for was forced on that board: a datagram held outstanding on the real usrsock backend while control operations arrive. A query answered, a peer update was accepted, the stop reported a timeout instead of hanging, the held ciphertext was untouched, and a repeated down reaped it so the interface came back. What that still does not cover is IOB exhaustion, losing Wi-Fi mid-send, and sustained load. The older apps version on the two boards — telnet, HTTP, a seven-megabyte transfer, rekey and power-cycle recovery — still stands underneath.

32 — Verification: A Working Tunnel Is Not Evidence (22:47 → 25:15) (*core — do not cut*)

On slide: Three rows (same private key every boot / session keys that outlive the tunnel / a leak that shows after hours), each with "would ping notice?" → no.

Say:

- The previous slide was where it runs. This one is what a working tunnel cannot tell you.
- Take key generation. NuttX can provide /dev/urandom through an xorshift128 backend whose seed material is compile-time configuration. A board left on that path can hand out the same private key on every boot. The handshake works. The ping works. The demo works. The tunnel is worthless, and nothing I showed you so far would notice. So I reset the board three times and compared one generated key from each boot: three different keys, with the entropy-pool backend selected rather than xorshift128. That is a useful regression check, not a certification of entropy quality.
- Second, what survives bringing the interface down. Here I stopped reading the code and read the memory: find the device in the running process by matching the key I set, then check the secret fields by name, at offsets computed from the driver's own headers. The transport keys are fully populated while the tunnel is up and every byte is zero after down, and the handshake state is already gone while it is up, because it is wiped the moment the keys are derived. The static key is still there — on purpose, because up has to work again — and there is exactly one copy of it in the whole writable address space, before and after. That asymmetry is a decision, so it is now written in the driver's documentation rather than left for a reviewer to guess at.
- Third, the soak. A hundred and fifty-one rekeys forced past the point where sessions expire, ten endpoint moves, a hundred down-up cycles, and the resources sampled throughout: the buffer pool comes back whole, the live allocation count returns to exactly what it was at boot — not close, equal — and the receive thread's stack high-water mark is two and a half kilobytes and does not move after the first sample. That last number is worth having on its own: the default stack for that thread is six kilobytes, so the margin is real rather than assumed. What I am not claiming is a week of uptime, or that a simulator's allocator and 64-bit frames behave like the board's.

33 — A Design Call: The Timestamp Problem Has No Easy Answer (25:15 → 28:49) (*core — do not cut*)

On slide: What replay protection obliges, the four options (real-time clock + in-boot high-water mark / persist every time / durable range reservation / wait to catch up), and the measured pair: clock unset → no handshake in 75 s, clock set → 4.1 s.

Say:

- This is the one where I had to decide rather than test. Then I went and measured the decision.
- WireGuard's replay defence puts an obligation on the initiator: every handshake must carry a timestamp greater than anything that peer already accepted, and that has to hold across a reboot. The device is usually the one dialling out, and it is the one least likely to have a battery-backed clock.

- The reproduction said recovery took seventy-five seconds, and that number is misleading. The device has to climb back past the last handshake of its previous session, so the outage is as long as the previous uptime. Seventy-five seconds only because that session was short. Something that ran for a month cannot reconnect for a month. That kills the do-nothing option outright.
- Writing the timestamp every handshake means a flash write every couple of minutes, forever, on the path that most needs to stay quick. Reserving a range durably and spending it from RAM is the right long-term answer, and I have written it up — but it needs a storage backend that will state its ordering and durability guarantees, and I could not get one. Locally, hostfs sync calls a void function and returns OK, so a successful write in the simulator proves nothing about power loss. Shipping that would look solved without being solved.
- So: realtime plus a high-water mark within the boot, which is exactly what Linux and wireguard-go do, and never fall back to uptime. What I added is the part a desktop never needed, because a desktop always has a clock. Without an RTC, NuttX seeds the clock from a configured year, so if the clock is still below the year after it, it was never set — and the driver says so once, naming the consequence. It still issues the timestamp, because a peer meeting this key for the first time will accept it and refusing would break first pairing on exactly those boards. The failure I am turning into one line of console output is not a crash. It is a tunnel that quietly never comes back, on a device that is not on your desk.
- And I did measure it, on the board, because a design argument you cannot demonstrate is just an opinion. The trap is that the obvious test passes. Reset the board, ping it, and the tunnel is back in sixteen seconds — because your ping made the *peer* initiate, and a responder needs no timestamp of its own. The device can be completely unable to dial out and that test still goes green.
- So: send the board nothing at all, and watch its own view of the session. Two runs, same board, same peer, same image, one variable. Clock as it boots: the driver warns, and there is no handshake after seventy-five seconds. Clock set from my laptop: the warning goes quiet, and the handshake completes in four point one seconds. That is both halves at once — the uncovered case is real on hardware, not just in a simulator, and the design is right wherever a clock exists. Fourteen stays open because I decided it should, not because I ran out of evidence.

34 — Agenda: ⑥ Demo (28:49 → 28:59)

Say (one line): Everything so far was engineering. Now, what it actually looks like.

Pause: point at the highlighted line, one beat.

35 — The Demo (28:59 → 30:29)

Say:

- The point of the demo is that there is nothing special to see.
- I telnet into the board through the tunnel and run commands, and I start a web server on the board and open it in a browser. The USB cable is unplugged; the board is on a power adapter. The peer is the official Windows client over home Wi-Fi. The board is just an ordinary host at the far end of an encrypted tunnel — which is exactly the goal.

Live: `uname -a / ifconfig (wg0 = 10.10.0.2) / wg show (handshake, transfer bytes) / ps (wg_rx running) / webserver &`, then `http://10.10.0.2/` in the browser. Show `wg show` before and after opening the page: the growing byte count is the evidence, not the terminal text.

Never on screen: the contents of `.config`, `kconfig-tweak` runs, build logs, `wg showconf` (prints the private key). The SSID and passphrase are in plain text in those places.

Fallback: the recording, youtu.be/1kyX2av5WG4. Switch at once if the live demo stalls.

36 — Agenda: ⑦ Toward Upstream (30:29 → 30:39)

Say (one line): Last chapter: what it takes to go from a demo to something maintainable, and what to carry out.

Pause: point at the highlighted line, one beat.

37 — Operability (30:39 → 31:13) (*cut candidate 3*)

Say:

- Moving from "it works in a demo" to "you could run it."
- Keys are no longer baked into the build — you generate and set them at run time, and they never end up in the firmware image. And for long runs I sample four independent signals every minute — is the router side alive, is the board on the network, is WireGuard carrying, is the application data flowing — so when it goes quiet I can say which layer failed instead of guessing.

38 — Portability (31:13 → 31:39) (*cut candidate 2*)

Say:

- The design decision from the start, validated.
- Isolating the OS behind a few functions was a bet. Here is the check, with the apps FLAT implementation: four architectures — x86, ARM Cortex-A7, Xtensa on the ESP32, and Cortex-M4F on SPRESENSE — with no code change to move between them. The kernel driver then adds sim and the rv-virt kernel build on top. The bet paid off.

39 — Contributing Back (31:39 → 32:15) (*cut first*)

Say:

- Closing the loop with this room's values.
- The contribution is staged: a small crypto PR first, for the nonce fix, because that stands on its own and helps everyone. Then the driver PR for the kernel device, and then the command PR for the user-space tool. Three signed candidate branches are published on my forks and the patch-form checks pass. The design discussion and upstream PRs are the next step; they are not open yet. Community over code means saying where the work actually is, then letting review change it.

If asked "is it merged?": not yet. These are submission candidates, not a claim that it is mergeable. See the Q&A file.

If time is short: skip the whole slide. The talk's argument (the pattern, the verification) does not rest on it; just keep the answer above ready for questions.

40 — Takeaways (32:15 → 33:14)

Say:

- Three things to take away.
- First, shallow tests pass and deep tests fail — so build your tests for depth: sustained traffic, real hardware, a real kernel build, not just a handshake. Second, a clean OS-abstraction boundary is worth the effort — four architectures and three versions with no changes. Third, porting into a project is a chance to improve it. One bug was in NuttX's own crypto, and that fix goes upstream on its own. The second I only found by insisting my board image be reproducible from a clean checkout: a stock release tag does not boot to a shell on this board, because before the RTC comes up the clock reads zero and the watchdog that would finish bringing the RTC up never fires. And a third was in my own driver, where only a real kernel build surfaced it; sim never would have.

41 — Thank You (33:14 → 33:22)

Say:

- Thank you. I am happy to take questions — about the kernel driver, the bugs, the ioctl ABI, or the hardware.

After: keep [coc-glasgow-qa.md](#) at hand, not a slide. The usual questions (rebase status, #14, IOB exhaustion, checkpatch) are answered there and on no slide.